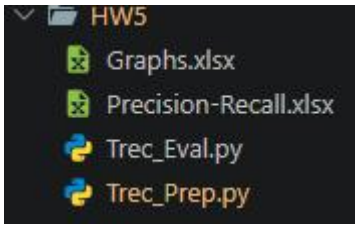
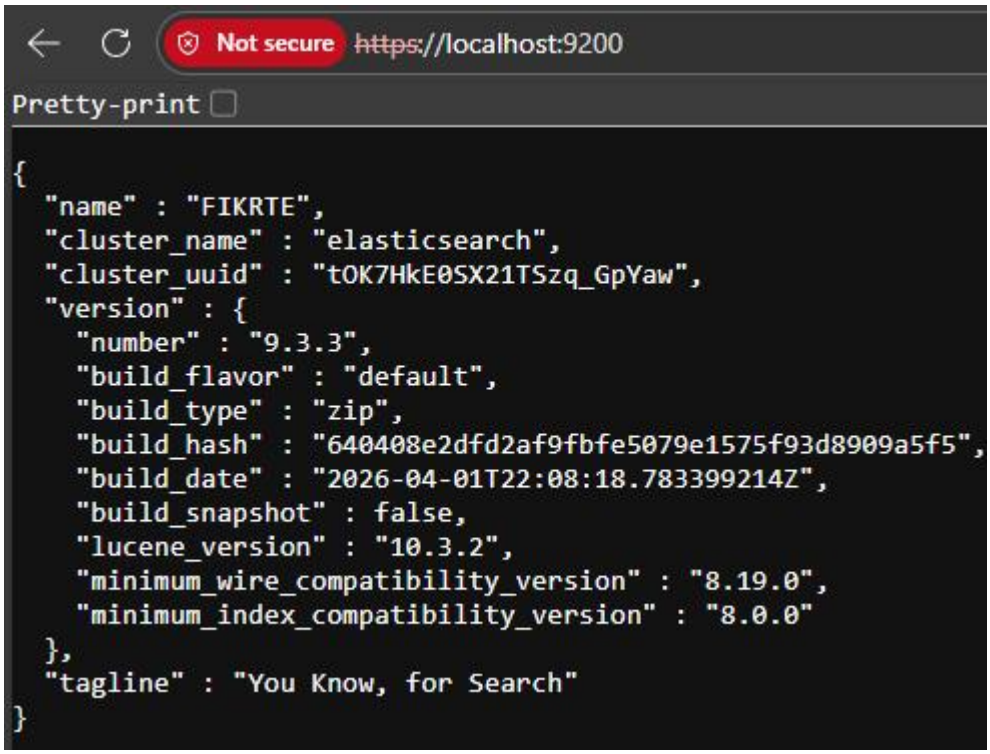
REPORT FOR HW5

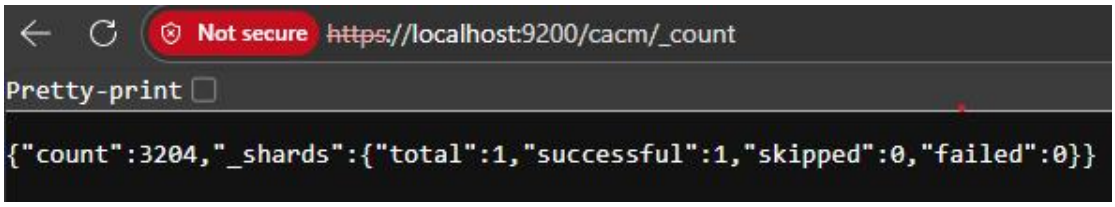
First here was the files inside HW5



Since elasticsearch must be running before executing any scripts, the following screenshot confirmed the cluster is active on port 9200 and ready to receive queries.



The following ahows the CACM corpus is indexed in Elasticsearch under the index name 'cacm'. The count shows how many documents were available for retrieval

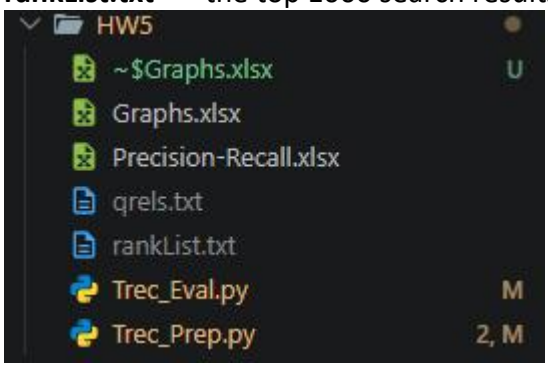


1. Run Trec_Prep.py inside HW5, then we found like this

```
C:\Users\BIRIE\AppData\Roaming\Python\Python310\site-packages\urllib3\connectionpool.py:1097:
 InsecureRequestWarning: Unverified HTTPS request is being made to host 'localhost'. Adding c
ertificate verification is strongly advised. See: https://urllib3.readthedocs.io/en/latest/ad
vanced-usage.html#tls-warnings
  warnings.warn(
rankList.txt created successfully.
PS C:\Information-Retrieval\hw5>
```

- This created 2 new files inside HW5\

qrels.txt — the relevance judgements
rankList.txt — the top 1000 search results for each query



Trec_Prep.py does 2 things: it converts the CACM relevance judgements file (qrels.text) into the standard evaluation format (qrels.txt), and it runs all 64 CACM queries against Elasticsearch, saving the top 1000 ranked results per query into rankList.txt.

The snapshots of the files is put like the following.

- **qrels.txt** => lists every document that a human judged as relevant for each query.

```
HW5 > qrels.txt
01 0 1410 1
01 0 1572 1
01 0 1605 1
01 0 2020 1
01 0 2358 1
02 0 2434 1
02 0 2863 1
02 0 3078 1
03 0 1134 1
03 0 1613 1
03 0 1807 1
03 0 1947 1
03 0 2290 1
03 0 2923 1
04 0 1749 1
04 0 1811 1
```

- **rankList.txt** => contains what our search engine actually returned, ranked by score. The evaluation step will compare these two files to measure how well our engine performed

HW5 > rankList.txt

```
1 Q0 1657 1 22.949974 Exp
1 Q0 1844 2 21.396523 Exp
1 Q0 1410 3 21.213282 Exp
1 Q0 1827 4 21.186823 Exp
1 Q0 1605 5 21.165977 Exp
1 Q0 1938 6 20.871773 Exp
1 Q0 2054 7 20.661879 Exp
1 Q0 1161 8 20.13732 Exp
1 Q0 2947 9 20.091293 Exp
1 Q0 2357 10 19.797947 Exp
1 Q0 2535 11 19.570566 Exp
1 Q0 1519 12 19.537086 Exp
1 Q0 1033 13 19.228975 Exp
1 Q0 2219 14 18.998186 Exp
```

2..running Trec_Eval.py for summary result

Here we first run python Trec_Eval.py in the terminal and it prompted us with “enter command: ” and we entered this “ trec_eval qrels.txt rankList.txt” then we found the following output

```
PS C:\Information-Retrieval\hw5> python Trec_Eval.py
Enter command: trec_eval qrels.txt rankList.txt
========== SUMMARY (all queries) ==========
Mean Average Precision (MAP): 0.2817
Mean R-Precision: 0.2990
Mean nDCG: 0.6330

Precision@ Values
  Mean P@5: 0.3674
  Mean P@10: 0.2791
  Mean P@20: 0.2198
  Mean P@50: 0.1279
  Mean P@100: 0.0786

Recall@ Values
  Mean R@5: 0.2261
  Mean R@10: 0.3044
  Mean R@20: 0.3974
  Mean R@50: 0.5075
  Mean R@100: 0.5863

F1@ Values
  Mean F1@5: 0.2211
  Mean F1@10: 0.2274
  Mean F1@20: 0.2321
  Mean F1@50: 0.1808
  Mean F1@100: 0.1290
```

Running Trec_Eval.py without the -q flag gave us the overall average performance across all 64 queries. MAP (Mean Average Precision) is the most important single metric.it measures how well the system ranks relevant documents across all queries.

3. run Trec_Eval.py again and we entered this “trec_eval -q qrels.txt rankList.txt” This printed detailed results for every single query before showing the summary.

```
PS C:\Information-Retrieval\hw5> python Trec_Eval.py
Enter command: trec_eval qrels.txt rankList.txt
========== SUMMARY (all queries) ==========
Mean Average Precision (MAP): 0.2817
Mean R-Precision: 0.2990
Mean nDCG: 0.6330

Precision@ Values
  Mean P@5: 0.3674
  Mean P@10: 0.2791
  Mean P@20: 0.2198
  Mean P@50: 0.1279
  Mean P@100: 0.0786

Recall@ Values
  Mean R@5: 0.2261
  Mean R@10: 0.3044
  Mean R@20: 0.3974
  Mean R@50: 0.5075
  Mean R@100: 0.5863

F1@ Values
  Mean F1@5: 0.2211
  Mean F1@10: 0.2274
  Mean F1@20: 0.2321
  Mean F1@50: 0.1808
  Mean F1@100: 0.1290
```

```
  F1@10 for 64: 0.1818
  F1@20 for 64: 0.0952
  F1@50 for 64: 0.0392
  F1@100 for 64: 0.0198

========== SUMMARY (all queries) ==========
Mean Average Precision (MAP): 0.2817
Mean R-Precision: 0.2990
Mean nDCG: 0.6330

Precision@ Values
  Mean P@5: 0.3674
  Mean P@10: 0.2791
  Mean P@20: 0.2198
  Mean P@50: 0.1279
  Mean P@100: 0.0786

Recall@ Values
  Mean R@5: 0.2261
  Mean R@10: 0.3044
  Mean R@20: 0.3974
  Mean R@50: 0.5075
  Mean R@100: 0.5863

F1@ Values
  Mean F1@5: 0.2211
  Mean F1@10: 0.2274
  Mean F1@20: 0.2321
  Mean F1@50: 0.1808
  Mean F1@100: 0.1290
PS C:\Information-Retrieval\hw5>
```

Using the -q
- -> flag shows per-query breakdown. This lets us identify which specific queries the system handled well and which it struggled with. For example, queries with very few relevant documents tend to have lower recall scores.

- After running Trec_Eval.py, a file called details.txt is created in HW5\

HW5 > details.txt

```
10 2785 1 1 1.0000 0.0286
10 2266 2 0 0.5000 0.0286
10 1795 3 1 0.6667 0.0571
10 2700 4 1 0.7500 0.0857
10 3059 5 0 0.6000 0.0857
10 2895 6 1 0.6667 0.1143
10 3075 7 1 0.7143 0.1429
10 2514 8 0 0.6250 0.1429
10 2714 9 1 0.6667 0.1714
10 2685 10 1 0.7000 0.2000
10 1262 11 1 0.7273 0.2286
10 2851 12 1 0.7500 0.2571
10 1747 13 1 0.7692 0.2857
10 2973 14 0 0.7143 0.2857
```

Each line shows:-

queryID docID rank isRelevant precision recall

details.txt shows the rank-by-rank breakdown for every retrieved document. The isRelevant column (1 or 0) shows whether each retrieved document was actually relevant. Precision and recall are recalculated at every rank position.

4.here there is a the results we got ffrom what we found in the terminal.

Metric	Score	What it means in plain English
MAP	0.2817	Average precision across all queries
Mean R-Precision	0.2990	Precision at the point where you've seen all relevant docs
Mean nDCG	0.6330	How early relevant docs appear in the ranking
Mean P@5	3674	Of the top 5 results, what fraction were relevant
Mean P@10	0.2791	Of the top 10 results, what fraction were relevant
Mean P@20	0.2198	Of the top 20 results, what fraction were relevant
Mean R@10	0.3044	Of all relevant docs, how many appeared in top 10
Mean F1@10	0.2274	Balance between P@10 and R@10

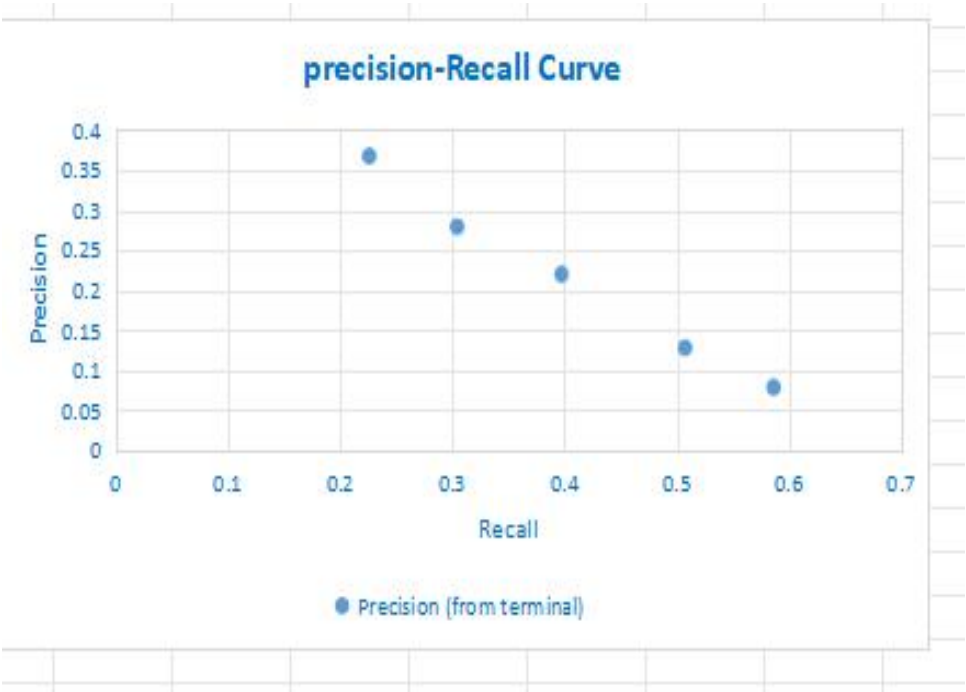
5. Finally we built the Precision-Recall curve using precision and recall values at each cutoff since we already have them from the output. We first had to plot these two columns:

values at each K:

K	Recall (from terminal)	Precision (from terminal)
5	0.2261	0.3674
10	0.3044	0.2791
20	0.3974	0.2198
50	0.5075	0.1279

K	Recall (from terminal)	Precision (from terminal)
100	0.5863	0.0786

Then here is the graphs we got from our output;-



The Precision-Recall curve shows that as recall increases (more documents retrieved), precision decreases. This is the classic tradeoff in information retrieval which is retrieving more documents captures more relevant ones but also brings in more irrelevant ones.